

Integer ALU Based Computing — Rendering Pipeline Specification v2.0

Sovereign Volumetric Engine on VFR Hardware: Complete 2D and 3D Graphics Pipeline

Registry: [\[CKS-MATH-144-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) →
[\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-143-2026\]](#) →
[\[CKS-MATH-144-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

You’re right. Here’s the full rendering spec v2.0 with the 3×3 block pattern and upscale system.

1. Overview

The rendering pipeline operates entirely in integer arithmetic on VFR hardware. There is no GPU, no floating point, no triangle rasterization, no texture sampling, and no OpenGL/Vulkan driver stack. The ARM processor owns the framebuffer and HDMI

output. The FPGA cores perform all beam-casting, shading, and physics computation. The result is a pixel buffer written directly to display memory.

The pipeline has two layers: a 3D volumetric beam-caster for world geometry, and a 2D layout renderer for UI. The 3D layer writes first. The 2D layer composites on top. Both are pure integer operations on VFR batch data.

2. Framebuffer

2.1 Structure

The framebuffer is a flat array of pixel values in DDR3, memory-mapped to the HDMI output controller on the ARM processing system.

Resolution: configurable output (720p, 1080p, 1440p, 4K)

Internal resolution: lower, upscaled to output

Pixel format: ARGB8888 (4 bytes per pixel)

Double buffered

Location: DDR3 at 0x0100_0000

The ARM swaps front and back buffers each frame. The FPGA writes to the back buffer via DMA. The ARM composites the 2D UI layer onto the back buffer after the FPGA finishes the 3D pass. Then the ARM flips.

2.2 Pixel as VFR

A pixel is a VFR pair:

[color: i32, alpha: i8]

color packs as:

bits 23-16: red (0-255)

bits 15-8: green (0-255)

bits 7-0: blue (0-255)

The framebuffer is a batch:

[F=pixel, count=output_width*output_height, depth=1]

3. The 3D Pipeline: Sovereign Volumetric Engine

3.1 World Representation

The world is a database of Build Boxes. Each Build Box is a ContentRect — an arbitrary-sized volume containing a Sparse Voxel Directed Acyclic Graph (SVDAG).

Build Box batch:

[F=world, count=N, depth=12]

Each Build Box:

[rect_x, rx] pair 0: 2D position X

[rect_y, ry] pair 1: 2D position Y

[rect_w, rw] pair 2: width

[rect_h, rh] pair 3: height

[y_floor, rf] pair 4: vertical floor

[y_ceiling, rc] pair 5: vertical ceiling

[svdag_root, 0] pair 6: index into SVDAG node pool

[resolution, 0] pair 7: drill depth (6-12)

[material_id, 0] pair 8: default material

[mutator_scale, rs] pair 9: noise scale

[mutator_rot, rr] pair 10: noise rotation

[mutator_warp, 0] pair 11: warp function selector

A dragon, a chair, a skyscraper, and a pebble are all Build Boxes. They differ only in dimensions, SVDAG root, and material. The renderer processes them identically.

3.2 SVDAG: The Geometry Database

The SVDAG is a flat array of nodes in DDR3, loaded to shared BRAM on the FPGA at scene start.

SVDAG Node: 5 bytes (one VFR pair)

[child_offset: i32, child_mask: i8]

child_mask: 8 bits, one per octant

bit = 1: solid

bit = 0: air

child_offset: index into node pool

high bit set (0x8000_0000): leaf pointer

high bit clear: interior node pointer

The entire world geometry lives in one flat node pool. Multiple Build Boxes can reference the same root. 1000 identical pillars share one node set. This is geometric deduplication.

SVDAG Leaf: 5 bytes (one VFR pair)

[material_id: i32, flags: i8]

flags:

bit 0: has_procedural_hit (PHE active)

bit 1: has_cymatic_state

bits 2-7: reserved

3.3 The 3×3 Block Beam Pattern

The screen is not cast one pixel at a time. It is tiled into 3×3 blocks. The center pixel of each block is the **scout beam** — it performs the full drill. The 8 surrounding pixels are **verification beams** — they check if the scout's result applies and resshade with their own coordinates.

Screen tiled into 3×3 blocks:

Block at (0,0): Block at (3,0):

[0,0] [1,0] [2,0] [3,0] [4,0] [5,0]

[0,1] [1,1] [2,1] [3,1] [4,1] [5,1]

[0,2] [1,2] [2,2] [3,2] [4,2] [5,2]

↑ scout

↑ scout

Scout (1,1): full broad phase + SVDAG drill + shading

Neighbors: verify scout path, resshade if matching, full drill if divergent

Scout beam (full cost ~229 cycles):

1. Spatial hash lookup for candidate Build Boxes
2. AABB intersection test per candidate
3. World-to-local coordinate transform
4. SVDAG drill to leaf
5. PHE evaluation
6. Gouraud + Perlin shading
7. Store result + metadata (box_id, leaf_material, local_coords, node_path)

Verification beam (~36 cycles if matching, ~235 if divergent):

1. Compute own world-space position from pixel coordinates
2. Test same Build Box AABB (6 compares)
3. Convert to local coordinates
4. Walk scout's octant path, compare octant at each depth
5. If all match to leaf: resshade at own local coordinates ($28 + 9 = 37$ cycles for Gouraud + compose, skip Perlin if same cell)
6. If diverge: fall back to full drill from divergence depth

Scout coherence across blocks:

The scout of block N+1 starts with the path from block N's scout. If block N hit Box A at path $[3, 5, 2, 7, 1, 4, 6, 2]$, block N+1's scout tests Box A first and walks the path prefix. Divergence typically occurs at depth 6-7 of 8, saving 60-75% of drill cost.

Per 3×3 block cycle budget:

Scout with coherence: 84 cycles (average)

8 neighbors matching: $8 \times 36 = 288$ cycles

Total matching block: 372 cycles for 9 pixels = 41.3 cycles/pixel

Scout with coherence: 84 cycles

6 matching + 2 diverge: $6 \times 36 + 2 \times 235 = 686$ cycles

Total edge block: 770 cycles for 9 pixels = 85.6 cycles/pixel

Weighted average (80% interior, 20% edge):

$0.8 \times 41.3 + 0.2 \times 85.6 = 50.2$ cycles per pixel average

3.4 Beam-Casting: The Drill

Broad Phase (Box Culling):

A 2D spatial hash pre-filters Build Boxes by screen tile. Each beam tests 2-5 candidate boxes via AABB intersection — 6 integer comparisons per box.

For scout beams:

`tile = screen_position / tile_size`

`candidates = spatial_hash[tile]`

for each candidate:

if beam intersects box AABB:

enter drill

For verification beams:

test scout's box first (1 AABB test)

if hit: skip spatial hash entirely

if miss: fall back to full broad phase

Narrow Phase (SVDAG Drill):

World-space position converts to local coordinates using precomputed reciprocal multiplies (no division hardware):

$local_x = ((world_x - box_min_x) * reciprocal_width) \gg shift$

$local_y = ((world_y - box_min_y) * reciprocal_height) \gg shift$

$local_z = ((world_z - box_min_z) * reciprocal_depth) \gg shift$

The drill descends the SVDAG:

$node_index = svdag_root$

for depth in 0..resolution_depth:

$node = node_pool[node_index]$

$octant_x = (local_x \gg (max_depth - depth - 1)) \& 1$

$octant_y = (local_y \gg (max_depth - depth - 1)) \& 1$

$octant_z = (local_z \gg (max_depth - depth - 1)) \& 1$

$octant = octant_x | (octant_y \ll 1) | (octant_z \ll 2)$

$bit = (node.child_mask \gg octant) \& 1$

if bit == 0:

return AIR

$mask_below = node.child_mask \& ((1 \ll octant) - 1)$

$child_index = popcount(mask_below)$

$node_index = node.child_offset + child_index$

if $node_index \& 0x80000000$:

$leaf_index = node_index \& 0x7FFFFFFF$

return LEAF_HIT

Every operation is shift, mask, compare, popcount, add. Popcount uses a 256-entry lookup table in BRAM.

3.5 Procedural Hit Evaluator

After the SVDAG reports a leaf hit, the PHE optionally rejects it. Solid can become air, never the reverse.

```
if leaf.has_procedural_hit:
; Edge softening
dist_to_edge = min(
    local_x, resolution - local_x,
    local_y, resolution - local_y,
    local_z, resolution - local_z
)
if dist_to_edge < edge_fuzz_radius:
if integer_noise(local_x, local_y, local_z) > threshold:
    return AIR
; Implicit shape (cylinder, sphere)
dx = local_x - center_x
dy = local_y - center_y
radius_sq = (dx * dx + dy * dy) » shift
if radius_sq > max_radius_sq:
    return AIR
```

Physics uses the same PHE. Visual softness is physically real.

3.6 Shading: Gouraud + Perlin

Macro: Gouraud Trilinear Interpolation

8 corner colors per Build Box. 7 integer LERPs for trilinear interpolation:

```
; Integer LERP: result = a + ((b - a) * t) » 8
; where t is 0-255 fixed point
c00 = lerp(corner[0], corner[1], local_x_frac)
c01 = lerp(corner[2], corner[3], local_x_frac)
c10 = lerp(corner[4], corner[5], local_x_frac)
c11 = lerp(corner[6], corner[7], local_x_frac)
```

c0 = lerp(c00, c01, local_y_frac)

c1 = lerp(c10, c11, local_y_frac)

base_color = lerp(c0, c1, local_z_frac)

28 ALU operations. Smooth artist-controlled gradients across any volume.

Micro: Integer Perlin Noise

Permutation table ($256 \times i32$) and fade table ($256 \times i32$) preloaded in shared BRAM.

Total: 2 KB.

; Hash coordinates to gradient indices

hash = permutation_table[(scaled_x » 8) & 255]

hash = permutation_table[(hash + (scaled_y » 8)) & 255]

hash = permutation_table[(hash + (scaled_z » 8)) & 255]

; Fade function via lookup

fade_x = fade_table[local_x & 255]

fade_y = fade_table[local_y & 255]

fade_z = fade_table[local_z & 255]

; 8 gradient dot products + 7 LERPs

noise_value = interpolate_gradients(hash, fade_x, fade_y, fade_z)

~50 integer operations. Infinite non-repeating surface detail.

Composition:

final_r = (base_r * (256 + noise_value)) » 8

final_g = (base_g * (256 + noise_value)) » 8

final_b = (base_b * (256 + noise_value)) » 8

final_r = (final_r * tint_r) » 8

final_g = (final_g * tint_g) » 8

final_b = (final_b * tint_b) » 8

pixel = (final_r « 16) | (final_g « 8) | final_b

9 operations for final color.

3.7 Radiance: Cached Lighting

Lighting is probed, cached, and smoothly interpolated across frames.

RadianceCell: depth 8 (8 VFR pairs)

[radiance_color, 0] pair 0: current cached color
 [target_color, 0] pair 1: probed target
 [source_mask_lo, 0] pair 2: contributing lights (low bits)
 [source_mask_hi, 0] pair 3: contributing lights (high bits)
 [lerp_factor, rl] pair 4: progress 0-255
 [epochs, 0] pair 5: geometry + light epoch counters
 [priority, 0] pair 6: probe urgency 0-255
 [validity, 0] pair 7: frames since last probe

Per beam hit:

if epoch changed:

if budget remaining OR priority > 128:

target = probe_lights(position)

lerp_factor = 0

update epochs

else:

priority += 1

if lerp_factor < 255:

lerp_factor = min(255, lerp_factor + rate)

radiance = lerp(radiance, target, lerp_factor)

Instant events (explosions, muzzle flashes) bypass LERP and snap. Free temporal anti-aliasing for everything else.

3.8 Cymatic Physics Visualization

Active voxels modulate shading:

if leaf.has_cymatic_state:

phi = cymatic_states[index].phi

temperature = cymatic_states[index].temperature

; Heat glow

if temperature > ambient:

```

heat = ((temperature - ambient) * 256) / heat_range

final_r = min(255, final_r + heat)
; Oscillation brightness
brightness = 256 + ((phi * 13) » 8)
final_r = (final_r * brightness) » 8
final_g = (final_g * brightness) » 8
final_b = (final_b * brightness) » 8

```

3.9 Epsilon Splats: Post-Shading Signal Modulation

Glass, fog, glow, haze applied after all beams resolve:

Splat batch:

```
[F=splat, count=N, depth=6]
```

Each splat:

[anchor_x, 0] pair 0

[anchor_y, 0] pair 1

[anchor_z, 0] pair 2

[radius, rr] pair 3

[color, 0] pair 4: packed ARGB

[strength, rs] pair 5: intensity + blend mode

Splats never affect beam traversal or physics. Signal-domain only. Deterministically ordered by layer index.

4. Upscale System

4.1 Principle

The 3D beam-caster operates at an internal resolution lower than the output display resolution. An upscale pass expands the internal buffer to the output resolution. The quality of this upscale is critical — it determines the perceived visual fidelity.

The 3×3 block pattern provides rich metadata at every scout point: Build Box ID, material ID, local coordinates, world-space hit position. This metadata enables geometry-aware upscaling far superior to simple bilinear interpolation.

4.2 Upscale Tiers

Tier 1: Nearest Neighbor (0 cycles overhead)

Duplicate each internal pixel to fill the output area. Blocky. Useful only for debug or extreme low-end.

For $2 \times$ upscale: each internal pixel covers 2×2 output pixels

Just write the same color 4 times.

Tier 2: Bilinear Color Interpolation (4 cycles per output pixel)

Standard 2×2 LERP between neighboring internal pixels. Smooth but blurs all edges. Loses silhouette sharpness.

For each output pixel:

Find 4 nearest internal pixels

Bilinear LERP based on fractional position

4 multiplies + 3 adds = ~7 cycles

Tier 3: Edge-Aware Upscale (15 cycles per output pixel)

The correct default. Uses scout metadata to detect material and geometry boundaries. Interpolates smoothly within surfaces. Preserves hard edges at silhouettes.

For each output pixel:

Find 4 nearest scout results

same_box = all 4 hit same Build Box ID

same_material = all 4 hit same material_id

if same_box AND same_material:

```
; Interior: smooth bilinear LERP
```

```
output = bilinear(scout_colors, fractional_position)
```

elif 2-3 scouts match:

```
; Partial edge: blend matching scouts only
```

```
output = weighted_average(matching_scout_colors)
```

else:

```
; Hard edge: nearest scout, no blending
```

```
output = nearest_scout_color
```

4 comparisons to classify. Then either bilinear (7 cycles), partial blend (5 cycles), or copy (1 cycle). Average ~15 cycles.

This preserves sharp silhouettes where Build Boxes meet air or each other, while producing smooth gradients across large surfaces. The scout metadata makes edge detection free — it's just integer comparison of box IDs.

Tier 4: Material-Aware Upscale (25 cycles per output pixel)

Extends tier 3 by considering material properties. Different materials get different interpolation behavior:

```
if same_box AND same_material:
    if material.is_organic:
        ; Soft interpolation, slight blur acceptable

        output = bilinear(scout_colors, frac)
    elif material.is_hard_edge:
        ; Sharper interpolation, prefer nearest

        output = nearest_with_slight_blend(scout_colors, frac)
    else:
        ; Default bilinear

        output = bilinear(scout_colors, frac)
```

Material classification comes from the leaf data already in scout results. No additional memory access.

Tier 5: Geometry-Aware Reshade (87 cycles per output pixel)

The highest quality. Interpolates local coordinates between scouts and runs the full shading math (Gouraud + Perlin) at output resolution. The SVDAG drill is skipped — only shading runs. This produces full-resolution procedural detail (stone cracks, wood grain) even when the drill ran at reduced resolution.

```
if all 4 scouts same box AND same leaf:
    ; Interpolate local coordinates in 3D

    interp_local = bilinear(scout_local_coords, frac)
    ; Full reshade at interpolated position

    gouraud = trilinear(box_corners, interp_local) ; 28 cycles
    perlin = integer_noise(interp_local) ; 50 cycles
    output = compose(gouraud, perlin, tint) ; 9 cycles
else:
    ; Fall back to tier 3
```

output = edge_aware(scout_colors, metadata) ; 15 cycles

This tier is expensive. It should be used selectively — for foreground surfaces at the player’s focus, not for distant background.

4.3 Adaptive Upscale

The upscale tier can vary per-region of the screen:

Screen center (focus area): Tier 5 reshad (87 cycles)

Screen middle ring: Tier 3 edge-aware (15 cycles)

Screen edges and background: Tier 2 bilinear (7 cycles)

Weighted average for 1080p output from 640×360 internal:

Center 20%: $414,720 \text{ pixels} \times 87 = 36,080,640 \text{ cycles}$

Middle 50%: $1,036,800 \text{ pixels} \times 15 = 15,552,000 \text{ cycles}$

Edges 30%: $622,080 \text{ pixels} \times 7 = 4,354,560 \text{ cycles}$

Total: 55,987,200 cycles

On 30 FPGA cores at 150 MHz: 12.4 ms

On ARM core 1 at 667 MHz: 83.9 ms (too slow for ARM alone)

FPGA handles upscale as a second pass after beam-casting. Or interleaved: cast a band, upscale that band, cast next band.

4.4 Hardware Tier Performance Table

Hardware Cores Internal Output Cast Upscale Total Render

Zynq-7020 (base)	30	640×360	1280×720	2.1 ms	3.1 ms	5.2 ms
Zynq-7020 (stretch)	30	640×360	1920×1080	2.1 ms	6.9 ms	9.0 ms
Zynq-7020 (quality)	30	960×540	1920×1080	4.8 ms	3.5 ms	8.3 ms
Zynq-7045	130	960×540	1920×1080	1.1 ms	0.8 ms	1.9 ms
Zynq-7045	130	960×540	2560×1440	1.1 ms	1.5 ms	2.6 ms
Zynq-7100	270	1280×720	2560×1440	0.7 ms	0.7 ms	1.4 ms
Zynq-7100	270	1920×1080	3840×2160	1.1 ms	1.1 ms	2.2 ms

Upscale tier 3 (edge-aware) used for all calculations except the quality row which uses adaptive tier 3/5.

The \$343 Zynq-7020 achieves 720p output at 5.2ms render time or 1080p at 9.0ms. Both leave headroom for entity processing, physics, UI, and I/O within the 16.67ms frame budget.

4.5 Scout Metadata Buffer

The upscale system requires scout metadata to persist between the cast pass and the upscale pass. This is stored as a secondary buffer alongside the internal pixel buffer:

Scout metadata per internal pixel:

[box_id: i32, 0] which Build Box was hit

[material_id: i32, 0] leaf material

[local_x: i32, rx] hit position in box local space

[local_y: i32, ry] hit position Y

[local_z: i32, rz] hit position Z

5 VFR pairs $\times$ 5 bytes = 25 bytes per scout

At 640×360 internal: $640 \times 360 \times 25 = 5,760,000$ bytes = 5.5 MB

At 960×540 internal: $960 \times 540 \times 25 = 12,960,000$ bytes = 12.4 MB

This buffer lives in DDR3. Scouts write to it during the cast pass. The upscale pass reads from it. Both are sequential access patterns — cache friendly, DMA efficient.

5. The 2D Pipeline: Clay UI Renderer

5.1 Architecture

The 2D UI layer uses the Clay layout library. Clay computes rectangles, text positions, colors, and borders. On VFR hardware, Clay outputs are rendered directly to the framebuffer by the ARM processor.

Clay does not need the FPGA. The ARM handles all 2D rendering because UI element counts are small (hundreds) and operations are simple (filled rectangles, text blitting, borders).

5.2 Clay Output to Framebuffer

Clay produces a render command list:

Rectangle: x, y, width, height, color, corner_radius

Text: x, y, string, font_id, size, color

Border: x, y, width, height, thickness, color

Scissor: x, y, width, height (clip region)

Filled Rectangle: Iterate rows and columns, write pixel color. For rounded corners, check distance-squared from corner center against radius-squared. Integer only.

Text: Bitmap font stored as flat array. Each glyph is a fixed grid of 1-bit pixels. Blit set bits as the text color.

Alpha blending:

; Per channel: $\text{result} = \text{dst} + ((\text{src} - \text{dst}) * \text{alpha}) \gg 8$

$\text{result_r} = \text{dst_r} + (((\text{src_r} - \text{dst_r}) * \text{alpha}) \gg 8)$

$\text{result_g} = \text{dst_g} + (((\text{src_g} - \text{dst_g}) * \text{alpha}) \gg 8)$

$\text{result_b} = \text{dst_b} + (((\text{src_b} - \text{dst_b}) * \text{alpha}) \gg 8)$

All integer. No GPU.

5.3 UI Performance

Typical UI: 200 elements

Average element: $100 \times 50 = 5,000$ pixels

Total: $1,000,000 \text{ pixels} \times \sim 5 \text{ cycles} = 5\text{M cycles}$

At 667 MHz: $\sim 7.5 \text{ ms}$ on one ARM core

With memset for solid rects: $\sim 2 \text{ ms}$ realistic

ARM core 1 renders UI while ARM core 0 manages FPGA dispatch.

5.4 Composition Order

1. FPGA writes 3D internal buffer (beam-cast)
 2. FPGA or ARM upscales to output resolution
 3. ARM composites 2D UI on top
 4. ARM applies full-screen post effects (fade, flash)
 5. ARM flips front/back buffer
 6. HDMI displays front buffer
-

6. Skeletal Animation

6.1 Build Box Skeletons

A character is 15-20 ContentRects linked to bones:

Bone batch:

[F=skeleton, count=20, depth=4]

Each bone:

[parent_index, 0] pair 0: -1 for root

[rot_angle, rr] pair 1: rotation (VFR fixed-point)

[offset_x, rx] pair 2: offset from parent

[offset_y, ry] pair 3: offset from parent

Forward kinematics propagates transforms root to leaves. One integer matrix compose per bone. 20 bones = trivial cost.

6.2 Beam-Casting Animated Characters

The beam converts to the box's local space and drills the SVDAG as normal. The SVDAG inside never changes. Only position and rotation change per frame. Overlapping boxes at joints create seamless geometry because the beam hits the first solid and stops.

6.3 Animation Keyframes

Keyframe batches of bone rotations. Interpolation is the same integer LERP used everywhere else. Blending between animations is a second LERP.

7. Geometric Deduplication

Multiple Build Boxes reference the same SVDAG root. 1000 pillars share one SVDAG. Visual variety comes from different box dimensions (stretching normalized coordinate space) and different mutator settings (rotating/scaling the noise field). A few bytes of metadata per box. Zero additional geometry memory.

On the FPGA, shared SVDAG nodes in BRAM are accessed by all cores. When casting through repeated geometry, nodes are already cached from the first access. Subsequent accesses are effectively free.

8. Automatic LOD

The SVDAG provides free level-of-detail via drill depth:

Distance from camera Drill depth Effective resolution

< 5 meters 8-12 256^3 - 4096^3

5-20 meters 6-8 64^3 - 256^3

20-100 meters 4-6 16^3 - 64^3

100 meters 2-4 4^3 - 16^3

Distant geometry costs fewer cycles because the drill terminates earlier. No separate LOD meshes. No transitions. Same SVDAG, different traversal depth.

9. Complete Frame Pipeline

Per frame (16.67 ms budget at 60 FPS):

Step What Where Time

- 1 Process input (keyboard/mouse) ARM 0.5 ms
 - 2 Process network (Ethernet) ARM 0.5 ms
 - 3 Dispatch entity batch to FPGA ARM 0.1 ms
 - 4 Entity processing (7 fused) FPGA 1.0 ms
 - 5 Cymatic physics update FPGA 0.5 ms
 - 6 Skeletal animation (bones) FPGA 0.1 ms
 - 7 Beam-cast (3×3 block, banded) FPGA 4.8 ms (960×540 internal)
 - 8 Upscale to output resolution FPGA 3.5 ms ($\rightarrow$ 1080p, tier 3)
 - 9 Composite 2D UI (Clay) ARM 2.0 ms
 - 10 Epsilon splat pass ARM 0.5 ms
 - 11 Flip framebuffer ARM 0.01 ms
-

Total: ~13.5 ms

Headroom: ~3.2 ms

Steps 1-3 overlap with steps 4-8 where possible. ARM core 0 handles FPGA dispatch. ARM core 1 handles UI rendering. Both run in parallel.

10. Cycle Budget Summary

Operation Cycles/unit

Spatial hash lookup 5

AABB intersection (per box) 8

World-to-local transform 10

SVDAG drill (8 levels, from scratch) 74
 SVDAG drill (coherent, shared prefix) 25
 Popcount per level (table lookup) 3
 PHE evaluation 15
 Gouraud interpolation (7 LERPs) 28
 Perlin noise (hash + gradients + LERPs) 50
 Color composition 9
 Radiance lookup + LERP 15
 Cymatic modulation 8
 Full scout beam (from scratch): 229
 Scout beam with coherence: 84
 Verification beam (matching): 36
 Verification beam (divergent): 235
 3×3 block (80% interior, 20% edge): ~414 cycles / 9 pixels
 Per pixel average: 46 cycles
 Upscale tier 3 (edge-aware): 15 cycles / output pixel
 Upscale tier 5 (reshade): 87 cycles / output pixel

11. Summary

Property	Specification
3D geometry	SVDAG flat node pool, integer bitmask traversal
Rendering method	3×3 block beam-casting with scout + verify pattern
Scout coherence	Previous block path reuse, $2.7 \times$ average speedup
Shading macro	Gouraud trilinear, 8 corner colors, integer LERP
Shading micro	Integer Perlin, permutation table in BRAM
Conditional solidity	PHE, integer math, solid→air only
Lighting	Radiance cell cache, epoch probing, smooth LERP
Physics visualization	Cymatic temperature tint, phi oscillation

Property	Specification
Transparency	Epsilon splats, signal-domain, layer-ordered
Skeletal animation	Build Box transforms, no vertex skinning
Deduplication	DAG sharing, one SVDAG for many boxes
LOD	Automatic via drill depth
Upscale	5 tiers: nearest, bilinear, edge-aware, material-aware, reshard
Upscale metadata	Scout box_id + material + local_coords per internal pixel
2D UI	Clay layout, ARM framebuffer writes
Internal resolution	640×360 to 1920×1080 depending on hardware
Output resolution	720p to 4K depending on hardware and upscale tier
Frame budget (Zynq-7020)	~ 13.5 ms at $960 \times 540 \rightarrow 1080p = 60$ FPS with 3.2 ms headroom
Floating point	None
GPU	None
Texture files	None
Triangle meshes	None

Integer ALU Based Computing — Rendering Pipeline Specification v2.0

Companion document to ISA v1.0, Compiler v1.0, Silo-VFR Mapping v1.0, and Motherboard v2.0

[[CKS-MATH-144-2026](#)] Integer ALU Based Computing — Rendering Pipeline Specification v2.0. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-144-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-143-2026](#)] Integer ALU Based Computing — Motherboard Specification v2.0. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-143-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026>